Java Backend Interview Prep - Multithreading and Miscellaneous

This document answers the Multithreading and Miscellaneous topics listed in the source interview sheet.[file:1]

Multithreading

1) Thread vs Runnable vs Callable

- `Thread`: concrete class representing a thread.
- `Runnable`: task with `run()` and no return value.
- `Callable<V>`: task with return value and checked exception support.[file:1]

```
Runnable r = () -> System.out.println("run");
Callable<Integer> c = () -> 42;
```

2) Ways to create threads

- Extend `Thread`.
- Implement `Runnable`.
- Use lambda for `Runnable`.
- Submit `Runnable`/`Callable` to `ExecutorService`.
- Use `CompletableFuture` for async composition.

3) synchronized keyword

`synchronized` protects a critical section using an intrinsic monitor lock. Only one thread can hold that monitor at a time.[file:1]

Under the hood, bytecode uses monitor enter/exit semantics for blocks and method-level synchronization.

4) Synchronized method vs block

- Method-level synchronization locks for the whole method.
- Block-level synchronization locks only a smaller critical section.

Prefer synchronized blocks when you want narrower lock scope and better throughput.

5) Instance vs static synchronized

- `synchronized` instance method locks on `this`.
- `static synchronized` method locks on `ClassName.class`.

So two threads can run one static synchronized method and one instance synchronized method at the same time because the locks differ.

6) Can constructors be synchronized?

No. Constructors are not inherited and object construction semantics make `synchronized` on constructors meaningless. Use synchronized blocks or factory methods if coordination is needed.[file:1]

7) What happens when a thread calls wait()?

`wait()` releases the monitor of that object and moves the thread to waiting state until notified and it can reacquire the lock.[file:1]

8) Race condition vs visibility problem

- **Race condition**: wrong result due to non-atomic interleaving.
- **Visibility problem**: one thread does not see another thread's latest write.

`volatile` solves visibility, not compound atomicity.

9) Daemon thread

Daemon threads support background work and do not keep the JVM alive. You must call `setDaemon(true)` before starting the thread; after start, it throws `IllegalThreadStateException`. [file:1]

10) wait(), notify(), notifyAll()

These methods are on `Object` because every object can act as a monitor lock in Java.[file:1]

11) ExecutorService and thread pools

Common pools:

- Fixed thread pool
- Cached thread pool
- Single thread executor
- Scheduled thread pool
- Work-stealing pool

Considerations:

- CPU vs I/O-bound workload

- Queue size
- Rejection policy
- Naming threads
- Graceful shutdown

12) wait() vs sleep()

- `wait()` releases monitor and must be called in synchronized context on an object.
- `sleep()` pauses the current thread without releasing locks.

13) Lock interface vs synchronized

`Lock` gives explicit `lock()`/`unlock()`, tryLock, interruptible locking, fairness options, and condition variables. `synchronized` is simpler and less error-prone.

14) ReentrantLock

A reentrant lock lets the same thread acquire the same lock multiple times without deadlocking itself.

```
ReentrantLock lock = new ReentrantLock(true); // fair lock
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

15) ThreadLocal

`ThreadLocal` provides thread-confined data, useful for request context, correlation IDs, or non-thread-safe formatter instances.

Be careful in thread pools; always clear values to avoid leaks.

16) Ensure T1 then T2

Use `join()`, `CountDownLatch`, or chaining through an executor.

```
Thread t1 = new Thread(() -> System.out.println("T1"));
Thread t2 = new Thread(() -> System.out.println("T2"));
t1.start();
t1.join();
t2.start();
```

17) CountDownLatch vs CyclicBarrier

- `CountDownLatch`: one-time coordination.
- `CyclicBarrier`: reusable barrier for a fixed number of threads.

Miscellaneous

18) Serializable, serialVersionUID, Externalizable

- `Serializable`: marker interface for default Java serialization.
- `serialVersionUID`: version identifier for compatibility checks.
- `Externalizable`: full control over serialization logic via `writeExternal/readExternal`.[file:1]

19) Marker interface

A marker interface has no methods and conveys metadata to the JVM/framework, e.g. `Serializable`, `Cloneable`.[file:1]

20) a = a + b vs a += b

`a += b` performs implicit cast if needed; `a = a + b` may fail compilation for narrower types.

```
short a = 1;
a += 1;      // ok
// a = a + 1; // compile error without cast
```

21) Overriding NPE with RuntimeException

Yes. `NullPointerException` is unchecked, and overriding methods may throw same or other unchecked exceptions.

22) Volatile array caveat

`volatile int[] arr` makes the array reference volatile, not each individual element. Writes to `arr[i]` are not automatically volatile-like.

23) clone method location

`clone()` is defined in `Object`. `Cloneable` is only a marker interface indicating cloning is allowed. [file:1]

24) Is Cloneable broken?

Many developers consider it awkward because of shallow-copy defaults, poor constructor interaction, and fragile inheritance behavior.

25) 5 * 0.1 == 0.5

Usually `true` in Java for this exact expression, but interviewers ask it to discuss floating-point representation. In general, direct equality checks with floating-point numbers are risky; use epsilon comparisons for computed values.[file:1]